

Generative classifiers: QDA, LDA and naive Bayes

Chapter resources

[Slides \(PDF\)](#)
[Open in Colab](#)
[Practice quiz](#)

In previous lectures, we have seen **discriminative models** (like Logistic Regression). These models have one goal: learn the *decision boundary* between classes. They directly model the conditional probability $P(Y = k|X = \mathbf{x})$.

Recall that in this stage Y is discrete and finite (our classes), while X can be either continuous or discrete, scalar or multidimensional.

In this lecture, we will learn about a completely different family of classifiers: **generative models**, and in particular, we will see the **Naive Bayes** model.

Generative models do not explicitly model the decision boundary. Instead, their goal is to **learn the “story” or “profile” of each class independently**. They aim to build a full probabilistic model of what each class *looks like*.

Technically, this means they model the **joint probability distribution** $P(X, Y)$. Recalling that $P(X, Y) = P(X|Y)P(Y)$, they do this by modeling two separate pieces:

1. **The Likelihood** $P(X = \mathbf{x}|Y = k)$: “What does a typical X look like *for this class*?”
2. **The Prior** $P(Y = k)$: “How common is *this class* in general?”

Once the model has learned these two things, it can be used for classification.

The Core Principle: Maximum A Posteriori (MAP)

Generative classifiers are built on the **Maximum A Posteriori (MAP)** principle. We want to find the class k that is *most probable* after we observe the data $\mathbf{x}$.

$$h(\mathbf{x}) = \arg \max_k P(Y = k|X = \mathbf{x})$$

Using Bayes' theorem, we can “flip” this equation to use the *generative* pieces we've learned:

$$h(\mathbf{x}) = \arg \max_k \frac{P(X = \mathbf{x}|Y = k)P(Y = k)}{P(X = \mathbf{x})}$$

We note that, if we are not interested in computing the actual probabilities $P(Y = k|\mathbf{x})$, but we only want to assign $\mathbf{x}$ to the most likely class, we can drop the evidence $P(X = \mathbf{x})$, which is independent of class k . Indeed we note that:

$$P(Y = k|\mathbf{x}) \propto P(X|Y = k)P(Y = k)$$

Which leads to the final MAP classification rule:

$$h(\mathbf{x}) = \arg \max_k \underbrace{P(X|Y = k)}_{\text{Likelihood}} \underbrace{P(Y = k)}_{\text{Prior}}$$

This approach is known as Maximum A Posteriori (MAP) classification as we aim to maximize the posterior probability.

In order to implement this principle, we need to compute the following two quantities:

- The likelihood $P(X|Y)$;
- The prior $P(Y)$;

We will now see how to compute each of these quantities.

The Prior $P(Y)$

$P(Y)$: this is the **prior probability of a given class**. If observing a class $Y = k$ is not very common, then $P(Y = k)$ will be small. We can use different approaches to estimate $P(Y = k)$:

- We can estimate $P(Y = k)$ by **considering the number of examples in the dataset**. For instance, if our dataset contains 800 non-spam e-mails and 200 spam e-mails and we set `Class 0 = Spam` and `Class 1 = Non Spam`, we can assume that $P(Y = 0) = 0.2$ and $P(Y = 1) = 0.8$.
- Alternatively, we could **study what is the proportion of examples in each class in the real world**. In the case of spam detection, we could ask a large sample of people how many e-mails they receive in average and how many spam e-mails they receive. These numbers can be used to define the prior probability.
- Another common choice, when we don't have enough information on the phenomenon is to **assume that all classes are equally probable**, in which case $P(Y = k) = \frac{1}{m}$ $\blacklozenge$, where m is the number of classes.

There are many ways to define the prior probability. However, it should be considered that this quantity should be interpreted in Bayesian terms. This means that, by specifying a prior probability, we are introducing our **degree of belief** on what classes are more or less likely in the system.

The Likelihood $P(X|Y)$

While estimating the prior is easy and estimating the evidence is not necessary for classification purposes (it would be indeed necessary if we were to compute probabilities), computing the likelihood term is less straightforward.

If we have M different classes, a general approach to estimate the likelihood consists in group all observations belonging to a given class $Y = k$ (let's call X_k the random variable of the examples belonging to this group) and estimate the probability $P(X_k)$.

If we repeat this process for every possible value of C , we have concretely estimated $P(X|Y = k)$ as:

$$P(X = x|Y = k) = P(X_k)$$

To estimate $P(X_k)$ we will generally need to make a few assumptions. Depending on such assumptions, we obtain different generative models.

The “Likelihood” Problem: From Ideal to Practical

The MAP rule is simple: $h(\mathbf{x}) = \arg \max_k P(X = \mathbf{x}|Y = k)P(Y = k)$.

The prior $P(Y = k)$ is easy to estimate (we just count). The *entire* challenge of generative modeling lies in estimating the **Likelihood**, $P(X = \mathbf{x}|Y = k)$.

How we do this depends on our features, and this is where we run (again) into the **Curse of Dimensionality**.

Guiding Example: Spam Detector

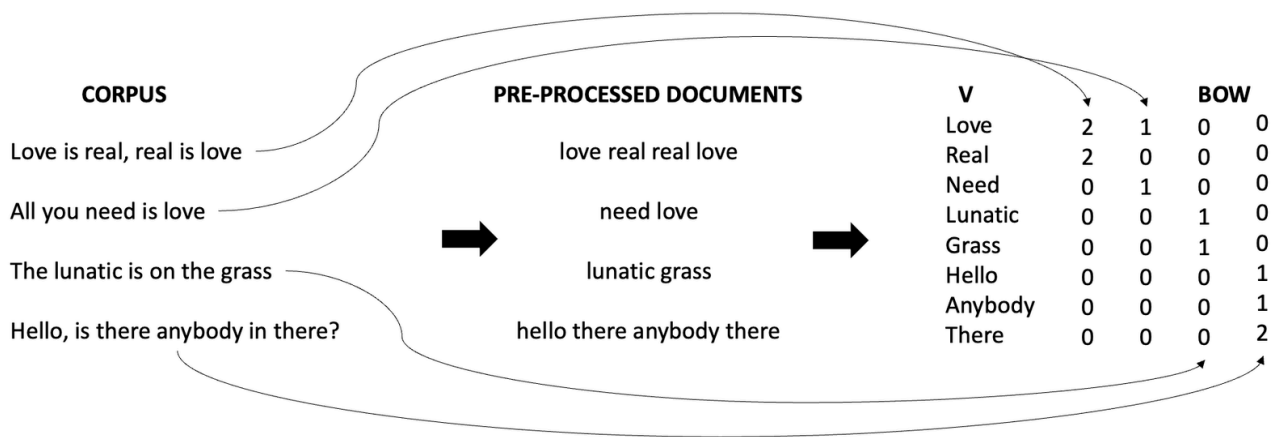
Spam detection is framed as a generative classification task, where the goal is to determine whether an incoming email is “spam” or “ham” based on its features.

Each email is represented as a high-dimensional vector, with features derived from word frequencies across a potentially massive vocabulary—often tens of thousands of words.

Since most emails contain only a small subset of these words, the resulting feature vectors are typically sparse.

This sparsity and dimensionality make estimating the likelihood function $P(X | Y = k)$ challenging, requiring strategic simplifications to model the data effectively.

The figure below shows how a text is converted into a vector of word counts. Note that in this process, we usually remove some less meaningful words such as articles and prepositions, which are usually called “stop words”:



The “Ideal” Model for Discrete Data (and its Limitations)

If our features are discrete (e.g., `Offer=Yes`, `Free=No`), we could try to model $P(X|Y)$ by building a giant contingency table for every single combination of features.

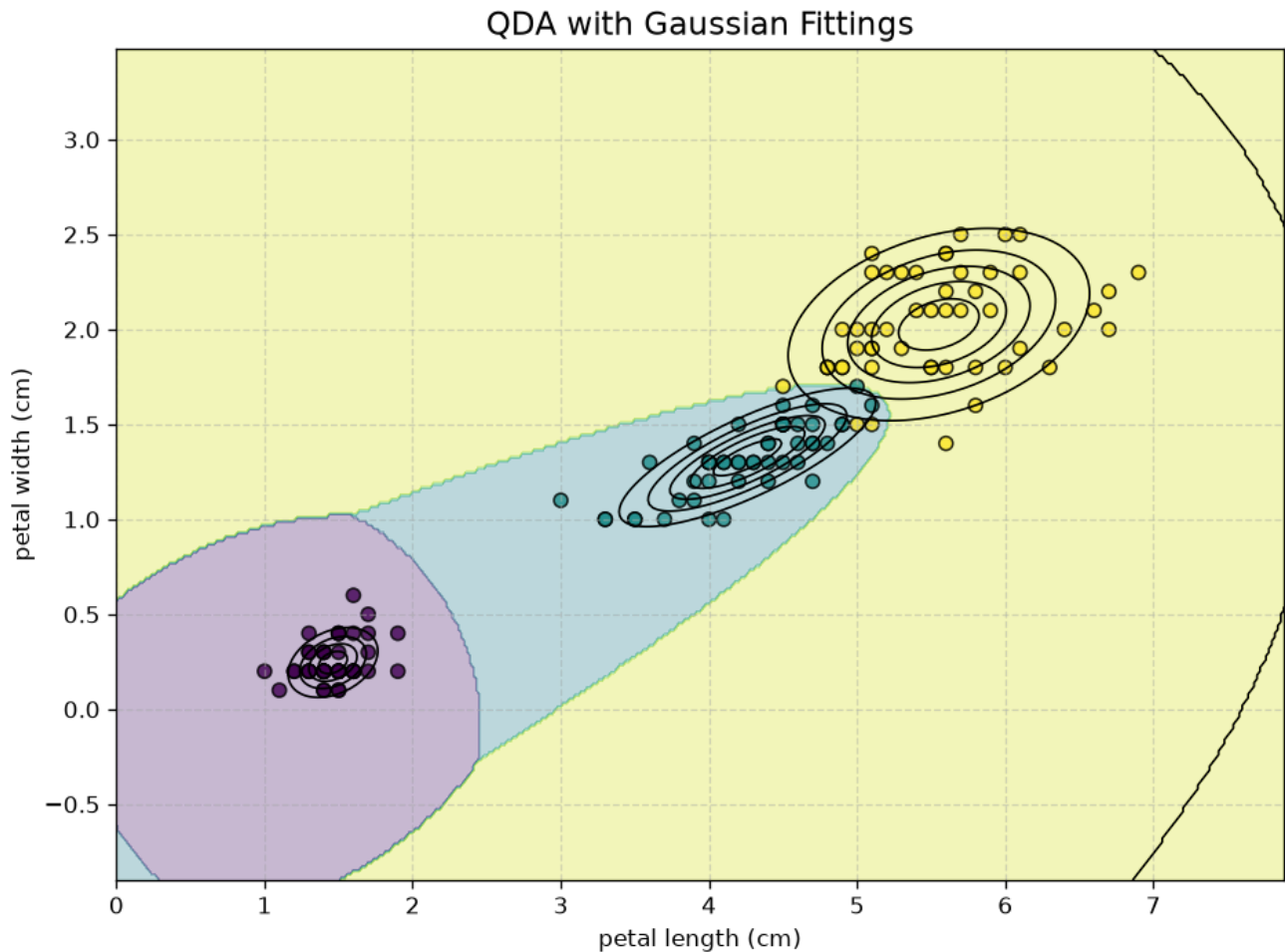
- **The Problem:** If we have 10,000 words (features) in our vocabulary, we would need to estimate the probability for $2^{10,000}$ possible combinations. This is computationally and statistically impossible.

The “Ideal” Model for Continuous Data: QDA

If our features are continuous (like the Iris dataset), we can’t build a table. The “ideal” approach is to model $P(X = \mathbf{x}|Y = k)$ as a **Multivariate Gaussian (Normal) Distribution**, $N(\mu_k, \Sigma_k)$.

This is **Quadratic Discriminant Analysis (QDA)**. It’s the most flexible Gaussian model.

- **Assumption:** Each class k has its own mean vector μ_k and its own, full covariance matrix Σ_k .
- **The “Nightmare”:** This is computationally a nightmare. To estimate a full covariance matrix Σ_k for d features, we must estimate $\sim \frac{d^2}{2}$ parameters. For 100 features, that’s $\approx 5,000$ parameters, per class. This requires massive amounts of data and overfits easily.
- **The Result:** A powerful, curved (quadratic) decision boundary.



QDA produces a powerful, curved (quadratic) decision boundary between classes, reflecting its flexibility in modelling distinct class distributions.

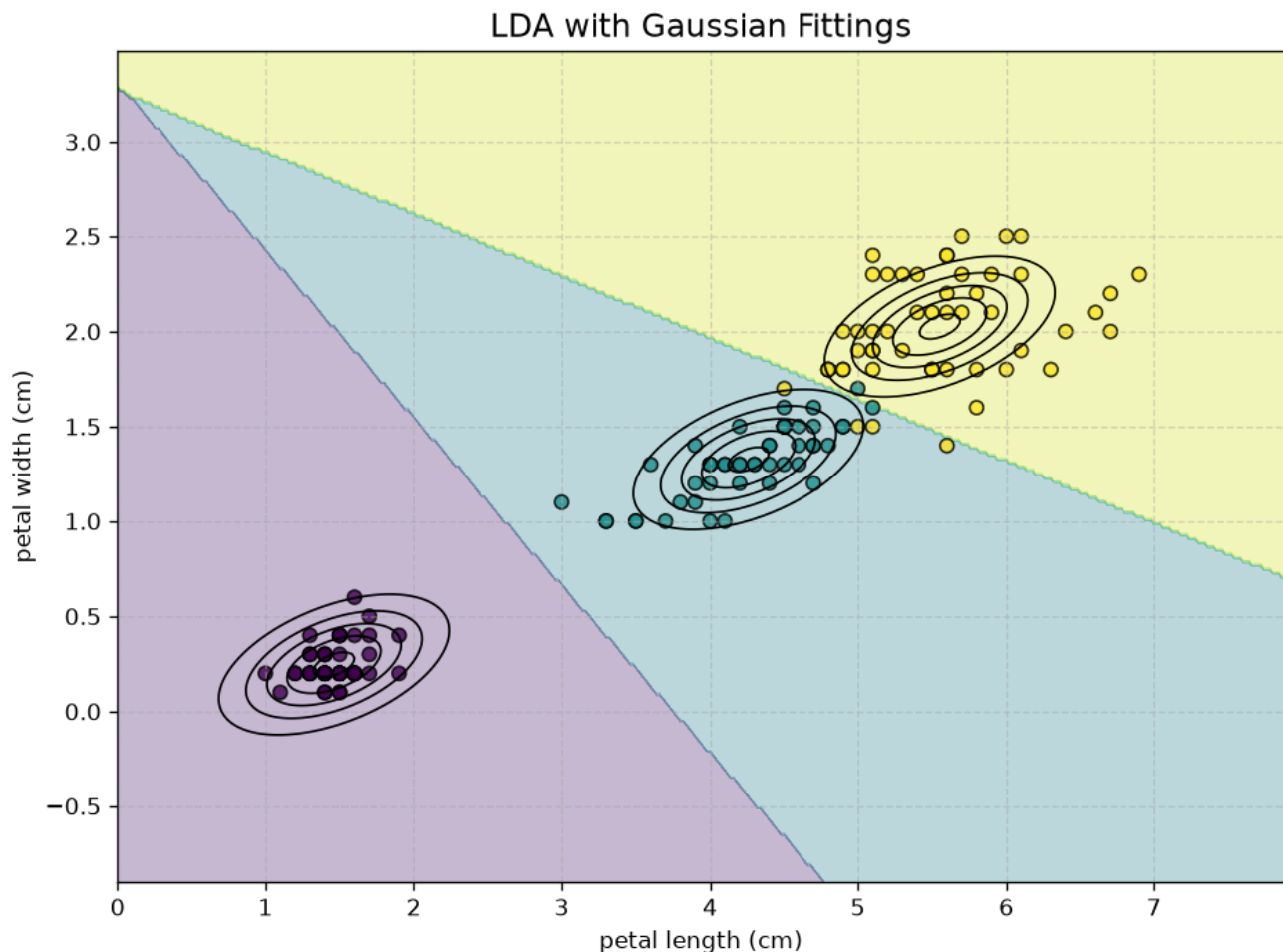
When we have enough data, QDA can lead to good results, but it can easily overfit when we don't have enough data.

Note that each class has its own Gaussian distribution and that distributions can be very different from each other.

The First Simplification: Linear Discriminant Analysis (LDA)

The “nightmare” of QDA leads to our first compromise: **Linear Discriminant Analysis (LDA)**.

- **Assumption:** We simplify the problem by assuming all classes *share* the **same covariance matrix** ($\Sigma_k = \Sigma$ for all k).
- **The Benefit:** This is much more stable. We only have to estimate *one* full Σ matrix, not k of them.
- **The “Nightmare” (Still):** We *still* have to estimate one full $\frac{d^2}{2}$ matrix. For text data with $d = 10,000$, this is still impossible.
- **The Result:** This assumption forces the math to simplify, and the decision boundary is always *linear*.



This critical assumption forces the mathematical derivations to simplify, resulting in a decision boundary that is always linear.

While less flexible than QDA's quadratic boundaries, LDA's linear approach is more robust and less prone to overfitting when data is scarce or dimensionality is high.

Note how now each class has the same “shape” (covariance). This makes the model less flexible and leads to a linear decision boundary.

Naive Bayes: The Pragmatic Solution

While LDA makes it easier to fit the model to the data, it can still be very computationally demanding.

Indeed, for **very high-dimensional data** ($d = 10,000$ features, like in text classification), estimating even *one* full covariance matrix $\hat{\Sigma}$ can be computationally and statistically challenging. This is an aspect of the **Curse of Dimensionality**.

This leads us to our final and most extreme simplification: **Naive Bayes**.

The “Naive” Assumption

Recall that our problem is estimating

$$P(X|Y)$$

Considering a multidimensional $X = (X_1, X_2, \dots, X_n)$, we may factorize the expression above as follows:

$$P(X|Y) = P(X_1, \dots, X_n|Y) = P(X_1|Y) P(X_2|X_1, Y) P(X_3|X_2, X_1, Y) \dots P(X_n|X_1, \dots, X_{n-1}, Y)$$

However, this is not a very helpful factorization as the terms $P(X_i|X_1, \dots, X_{i-1}, Y)$ are conditioned also on other features, which makes them not easy to model.

The Naive Bayes classifier makes one, very strong (“naive”) assumption:

It assumes that all features X_i are **conditionally independent** given the class $Y = k$.

This can be written as follows:

$$X_i \perp X_j \mid Y, \forall i \neq j$$

Spam Classification Example Let’s assume a spam classification example and imagine that

features X_i and X_j are related to the number of appearance of specific words in the current email represented by X . If X_i and X_j are **word counts**, then we are saying that if I take all non-spam e-mails, then the number of occurrences of a given word **does not influence** the number of occurrences of **another word**. This is **obviously not true in general!** For instance, there can be legitimate e-mails of different topics. If the e-mail is about a vacation, the words ‘trip’, ‘flight’, ‘luggage’ will appear often. Instead, if the e-mail is about work, the words ‘meeting’, ‘report’, ‘time’, will appear more often. This means that, within the same category (non-spam e-mails), the number of occurrences of a word (e.g., ‘trip’) may be related to the number of occurrences of another words (e.g., ‘flight’), which breaks the assumption of **conditional independence**. This is why this assumption is called **naïve assumption**. With this in mind, it should be considered that, despite such naïve assumption, the Naïve Bayes Classifier works surprisingly well in many contexts.

We know that:

$$X \perp Y \mid Z \Leftrightarrow P(X, Y \mid Z) = P(X \mid Z) P(Y \mid Z)$$

Hence, we discover that, under the assumption of conditional independence:

$$P(X_1, \dots, X_n | Y) = P(X_1 | Y) P(X_2 | Y) \dots P(X_n | Y)$$

So, we can re-write the MAP classification rule as:

$$f(\mathbf{x}) = \arg_k \max P(\mathbf{x}_1 | Y = k) P(\mathbf{x}_2 | Y = k) \dots P(\mathbf{x}_n | Y = k) P(Y = k)$$

Flavors of Naïve Bayes

The single $P(X_i | Y = k)$ terms are now easy to model, since X_i is mono-dimensional. In practice, depending on the considered problem, we can model these terms in different ways. Two common approaches, depending on the kind of data we have, are to use a **Gaussian distribution** or a **Multinomial distribution**.

When we use Gaussian distributions to model the $P(X_i | Y)$ terms, the classification method is called “**Gaussian Naïve Bayes**”. Similarly, if we consider a multinomial distribution, the classification method is called “**Multinomial Naïve Bayes**”.

Gaussian Naïve Bayes

Let us consider again our sex classification example based on height and weight. We will consider $X = [H, W]$, which are random variables representing heights and weights of subjects. If we assume that the data is approximately Gaussian, the probabilities $P(H|C)$ and $P(W|C)$ can be modeled with univariate (1D) Gaussian distributions. This is done by first obtaining four samples:

- H_1 : the heights of subjects when $C = 1$;
- W_1 : the weights of subjects when $C = 1$;
- H_0 : the heights of subjects when $C = 0$;
- W_0 : the weights of subjects when $C = 0$.

We hence model each sample as a 1D Gaussian distribution by computing a mean and a variance value from each of these samples to obtain four Gaussian distributions:

- $P(H = h | C = 0) = N(x; \mu_1, \sigma_1)$;
- $P(W = w | C = 0) = N(x; \mu_2, \sigma_2)$;
- $P(H = h | C = 1) = N(x; \mu_3, \sigma_3)$;
- $P(W = w | C = 1) = N(x; \mu_4, \sigma_4)$;

After this, we can apply the classification rule:

- The example (h, w) is classified as class 1 if

$$P(h|C=1)P(w|C=1)P(C=1) > P(h|C=0)P(w|C=0)P(C=0);$$

- The example (h, w) is classified as class 0 otherwise.

We can exemplify this process as follows:



Implications of the Naive Assumption in Gaussian Naive Bayes

Gaussian Naive Bayes assumes that features are conditionally independent given the class label. This

leads to a **diagonal covariance matrix** for each class Σ_k , meaning that the off-diagonal entries—representing feature correlations—are zero. Mathematically, this is expressed as:

$$\Sigma_k = \begin{bmatrix} \sigma_1^2 & 0 & 0 \\ 0 & \sigma_2^2 & 0 \\ 0 & 0 & \sigma_3^2 \end{bmatrix}$$

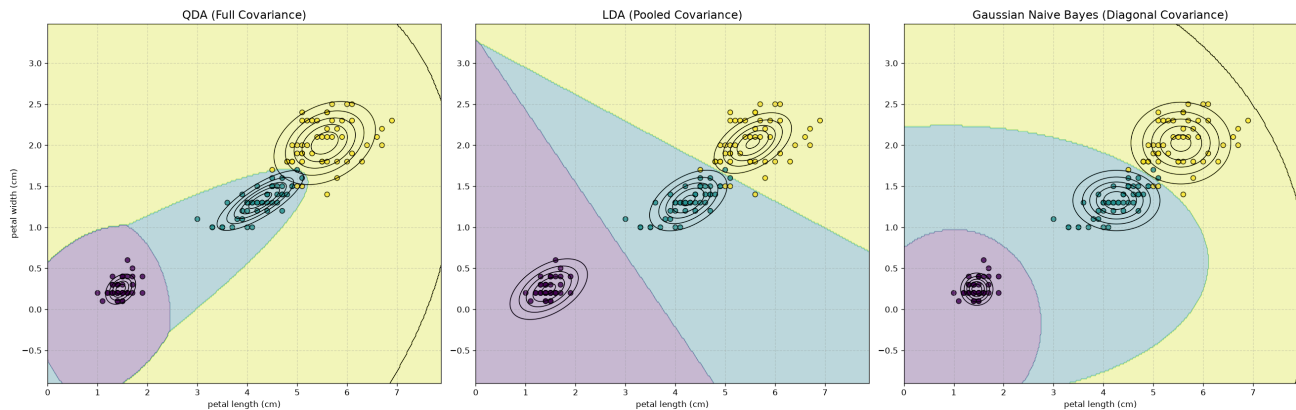
This structure implies that the model treats each feature as contributing independently to the likelihood $P(X | Y = k)$. As a result, the fitted Gaussian distributions are **axis-aligned**, meaning their contours are parallel to the coordinate axes. This simplifies computation and estimation, especially in high-dimensional spaces, but limits the model's ability to capture **intra-class correlations**.

Despite this simplification, Gaussian Naive Bayes allows each class to have its own diagonal

covariance matrix. That is, $\Sigma_k \neq \Sigma_j$ for $k \neq j$, which enables the model to produce **non-linear decision boundaries**. These boundaries arise from differences in class-specific variances, even though the distributions themselves remain axis-aligned. However, the inability to model feature interactions within a class remains a core limitation of the naive assumption.

Decision Boundaries and Comparison with QDA and LDA

Let's compare the decision boundaries of the three classifiers:



This figure perfectly summarizes the three models:

- **QDA (Left):** Most flexible. It learns a separate, *rotated* ellipse (full Σ_k) for each class. Its boundaries are quadratic.
- **LDA (Middle):** The compromise. It learns *one* pooled, *rotated* ellipse (full Σ) and uses it for all classes. Its boundaries are linear.
- **GNB (Right):** Most simple. It learns a separate, *axis-aligned* ellipse (diagonal Σ_k) for each class. Its boundaries are quadratic but can't capture any feature correlation (tilt).

Multinomial Naive Bayes (MNB)

In the previous section, we saw **Gaussian Naive Bayes (GNB)**, which is ideal for **continuous features** (like `petal_length`) that we can model with a Gaussian (bell curve).

When the input features are **discrete**, we must use a different “flavor” of Naive Bayes. The most common case is **text classification** (e.g., spam filtering).

If we represent text using **word counts** (a “Bag of Words” or BoW representation), our input features are a vector of discrete, natural numbers:

$$\mathbf{x} = (x_1, x_2, x_3, \dots, x_d)$$

where d is the size of our vocabulary (i.e., $d = |V|$) and $x_i \in \mathbb{N}$ is the number of times the i -th word in the vocabulary appears in the document $\mathbf{x}$.

Our goal is to model the likelihood, $P(\mathbf{x}|Y = k)$, which is:

$$P(x_1, x_2, \dots, x_d|Y = k)$$

If we imagine the process of “generating” a document as pulling n words out of a “bag” (our vocabulary V), we can model this probability $P(\mathbf{x}|Y = k)$ with a **Multinomial Distribution**. We will need to estimate a different Multinomial distribution for each class k .

The Multinomial Distribution: Remember that the multinomial distribution models the probability of obtaining exactly $(\mathbf{x}_1, \dots, \mathbf{x}_d)$ occurrences for each of the d possible outcomes (words), in a sequence of n independent experiments (total words), where each experiment follows a categorical distribution with probabilities $(p_{k1}, \dots, p_{kd})$.

In our case:

- $n = \sum_{i=1}^d x_i$: The total number of words in the document $\mathbf{x}$.
- d possible outcomes: The d words in the vocabulary.
- p_{ki} : The probability of randomly picking word i , given that we are in class k .

Fitting the Multinomial Distribution

Using the analytical form of the multinomial distribution of [chapter 5](#), we can write the full likelihood:

$$P(\mathbf{x}|Y = k) = \frac{(\sum_{i=1}^d x_i)!}{x_1! \dots x_d!} \cdot p_{k1}^{x_1} \cdot p_{k2}^{x_2} \cdot \dots \cdot p_{kd}^{x_d}$$

Now, we plug this into our MAP classification rule:

$$h(\mathbf{x}) = \arg \max_k P(Y = k) \cdot \frac{(\sum_{i=1}^d x_i)!}{x_1! \dots x_d!} \prod_{i=1}^d (p_{ki})^{x_i}$$

We can immediately simplify this. The big factorial term

$$\frac{(\sum_{i=1}^d x_i)!}{x_1! \dots x_d!}$$

...is a constant *with respect to* k . It only depends on the input $\mathbf{x}$, not the class. Since we are just trying to find the **arg max**, we can drop this term entirely, as it doesn't change which k gives the highest score.

This leaves us with a much simpler (and faster) classification rule:

$$h(\mathbf{x}) = \arg \max_k P(Y = k) \prod_{i=1}^d (p_{ki})^{x_i}$$

Estimating the Parameters

To use this model, we must estimate two sets of parameters from our training data: the priors

$P(Y = k)$ and the word likelihoods p_{ki} .

Estimating the Word Likelihoods p_{ki}

We estimate the probability of word i for class k by counting:

$$p_{ki} = \frac{\text{Total count of word } i \text{ in all documents of class } k}{\text{Total count of *all words* in all documents of class } k}$$

Using the Iverson bracket notation of [chapter 15](#), where $[y^{(j)} = k]$ is 1 if true and 0 otherwise:

$$p_{ki} = \frac{\sum_j [y^{(j)} = k] x_i^{(j)}}{\sum_{l=1}^d \left(\sum_j [y^{(j)} = k] x_l^{(j)} \right)}$$

- **Numerator:** Sums the counts of word i only in documents j from class k .
- **Denominator:** Sums the counts of *all* words ($l = 1$ to d) in *all* documents j from class k .

Estimating the Prior $P(Y = k)$

This is the same as before. We estimate the prior as the fraction of documents in our training set that belong to class k :

$$P(Y = k) = \frac{\sum_j [y^{(j)} = k]}{N}$$

Where N is the total number of documents.

Practical “Gotchas”: Two Critical Fixes

If we use the formulas above directly, our model will fail. We need to fix two major problems.

Problem 1: The Zero-Probability Problem

What happens if we get a new email that contains the word “crypto”? If our training data for `Class= 'Spam'` never contained the word “crypto,” then its estimated probability is: $p_{k, \text{'crypto'}} = 0$

When we classify this email, our MAP rule will multiply by this zero:

$$h(\mathbf{x}) \propto P(Y = \text{Spam}) \cdot \dots \cdot p_{k, \text{'crypto'}}^{x_{\text{'crypto'}}} \cdot \dots = P(Y = \text{Spam}) \cdot \dots \cdot 0 \cdot \dots = 0$$

Because of this one **zero**, the entire score for the “Spam” class becomes zero. The model breaks.

The Solution: Laplace (Add-One) Smoothing The fix is simple: we pretend we’ve seen every word in our vocabulary *at least one time* before we start. This is called **Laplace Smoothing**.

We add 1 (the “pseudo-count”) to the numerator, and we add $d = |V|$ (the vocabulary size) to the denominator to keep the probabilities summing to 1. This ensures no probability is ever zero.

$$p_{ki} = \frac{\left(\sum_j [y^{(j)} = k] x_i^{(j)}\right) + 1}{\left(\sum_{l=1}^d \sum_j [y^{(j)} = k] x_l^{(j)}\right) + |V|}$$

Problem 2: Arithmetic Underflow

Our final rule multiplies thousands of tiny probabilities together (e.g., $0.001 \times 0.0004 \times \dots$). This number gets so small, so fast, that the computer runs out of precision and rounds it to 0. This is called **arithmetic underflow**.

The Solution: Log-Probabilities Since the $\log$ function is monotonic (it doesn't change the `arg max`), we can maximize the *log-probability* instead.

Using the log-rule $\log(a \cdot b) = \log(a) + \log(b)$ and $\log(p^x) = x \cdot \log(p)$, we get:

$$h(\mathbf{x}) = \arg \max_k \left(\log P(Y = k) + \sum_{i=1}^d x_i \cdot \log p_{ki} \right)$$

This is the **final classification rule** that is actually implemented in `scikit-learn`. It's computationally stable, extremely fast (it's just a sum!), and is the foundation of modern text classification.

Lab Bake-Off: The Right vs. The Wrong Tool for the Job

We've now seen a "family" of generative models (QDA, LDA, GNB) and know about discriminative models like KNN and Logistic Regression.

Let's run a "bake-off" to see what happens when we apply these models to a **high-dimensional text classification** problem (spam filtering). This will clearly demonstrate *why* `Multinomial Naive Bayes (MNB)` is a standard tool for this task, and why the other models we've learned are the **wrong tool for the job**.

The Competitors:

1. Text-Native Models:

- `MultinomialNB` (Generative, for counts)
- `LogisticRegression` (Discriminative, works well with high-d text)

2. The "Wrong" Models:

- `KNeighborsClassifier` (Distance-based, suffers from Curse of Dimensionality)
- `GaussianNB` (Generative, assumes features are Gaussian, not counts)
- `LinearDiscriminantAnalysis` (Generative, needs a full covariance matrix)
- `QuadraticDiscriminantAnalysis` (Generative, needs *multiple* full covariance matrices)

The Workflow:

1. Load and clean the SMS Spam dataset.
2. Create a single **Train/Test Split**.

3. Build a `Pipeline` for each model. This is crucial:
 - `MNB` will use `CountVectorizer` (raw counts).
 - All other models will use `TfidfVectorizer` (normalized features).
 - The Gaussian models (GNB, LDA, QDA) *cannot* work with the default “sparse” matrix from a vectorizer, so we must build a custom “hack” to make them dense, which will highlight their inefficiency.
4. Fit all models, capture their accuracy and training time, and catch the errors for the models that will (and should) fail.
5. Compare all results in a final table and plot.

```
# --- 1. All Imports ---
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time

# For data loading
import io
import zipfile
import urllib.request

# Sklearn Workflow Tools
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Sklearn Feature Engineering
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.base import TransformerMixin, BaseEstimator # For our custom transformer

# Sklearn Models (The Competitors)
from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
```



Step 1: Load and Split the Data

First, we load the data from the UCI zip file, map the labels, and create our single, “locked-away” test set.

```
# 1. Load the dataset
try:
    zip_data = data_bytes("sms_spam_collection.zip")

    # 2. Unzip the file in memory
    with zipfile.ZipFile(io.BytesIO(zip_data)) as z:
        # 3. Read the specific file from the zip archive into pandas
        with z.open('SMSSpamCollection') as f:
            df = pd.read_csv(f,
                             sep='\t',
                             header=None,
                             names=['label', 'message'],
                             encoding='latin-1')

    # 4. Map labels to 0 (ham) and 1 (spam)
    df['label'] = df['label'].map({'ham': 0, 'spam': 1})

    print("--- Data Head ---")
    print(df.head())

    # 5. Define X and y
    X = df['message'] # The raw text
    y = df['label']   # The 0 or 1

    # 6. Create the Train/Test Split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                         random_state=42, stratify=y)
    print(f"\nTraining samples: {len(X_train)}, Test samples: {len(X_test)}")

except Exception as e:
    print(f"Error loading or processing data: {e}")
```

```
--- Data Head ---
   label      message
0      0  Go until jurong point, crazy.. Available only ...
1      0              Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...
```

```
Training samples: 3900, Test samples: 1672
```

Step 2: Define Pipelines and Custom Transformer

We need a custom transformer to convert the sparse output of `TfidfVectorizer` into a dense `numpy` array, which GNB, LDA, and QDA require. This is intentionally inefficient and will highlight the “nightmare” of using these models on high-dimensional data.



```
# Custom Transformer to convert sparse matrix to dense
class DenseTransformer(TransformerMixin, BaseEstimator):
    def fit(self, X, y=None):
        return self

    def transform(self, X, y=None):
        return X.toarray()

# --- 1. Define the models we want to test ---
models_to_test = {
    "Multinomial NB (Text-Native)": Pipeline([
        ('vectorizer', CountVectorizer(stop_words='english')),
        ('classifier', MultinomialNB())
    ]),

    "Logistic Regression (Discriminative)": Pipeline([
        ('vectorizer', TfidfVectorizer(stop_words='english')),
        ('classifier', LogisticRegression(random_state=42))
    ]),

    "K-Neighbors (Distance-Based)": Pipeline([
        ('vectorizer', TfidfVectorizer(stop_words='english')),
        ('classifier', KNeighborsClassifier(n_neighbors=5))
    ]),

    "Gaussian NB (Wrong Assumption)": Pipeline([
        ('vectorizer', TfidfVectorizer(stop_words='english')),
        ('to_dense', DenseTransformer()), # Must be dense for GNB
        ('classifier', GaussianNB())
    ]),

    "Linear Discriminant (LDA)": Pipeline([
        ('vectorizer', TfidfVectorizer(stop_words='english')),
        ('to_dense', DenseTransformer()), # Must be dense for LDA
        ('classifier', LinearDiscriminantAnalysis())
    ]),

    "Quadratic Discriminant (QDA)": Pipeline([
        ('vectorizer', TfidfVectorizer(stop_words='english')),
        ('to_dense', DenseTransformer()), # Must be dense for QDA
        ('classifier', QuadraticDiscriminantAnalysis())
    ])
}

# --- 2. Store the results ---
results = {}
```

Step 3: Run the Bake-Off

Now we loop, fit, and time each model. Watch what happens when we try to run the models that aren't designed for this kind of data.

```
for name, pipeline in models_to_test.items():
    print(f"--- Testing: {name} ---")
    start_time = time.time()

    try:
        # 1. Fit the pipeline
        pipeline.fit(X_train, y_train)

        # 2. Make predictions on the TEST set
        y_pred = pipeline.predict(X_test)

        # 3. Get the accuracy
        accuracy = accuracy_score(y_test, y_pred)

        # 4. Store results
        elapsed = time.time() - start_time
        results[name] = {'Test Accuracy': accuracy, 'Time (sec)': elapsed, 'Error': None}
        print(f" Success! Accuracy: {accuracy*100:.2f}%, Time: {elapsed:.2f}s")

    except Exception as e:
        # Catch models that fail to train
        elapsed = time.time() - start_time
        results[name] = {'Test Accuracy': 0.0, 'Time (sec)': elapsed, 'Error': str(e)}
        print(f" FAILED! Error: {e}")
```

```
--- Testing: Multinomial NB (Text-Native) ---
Success! Accuracy: 98.74%, Time: 0.05s
--- Testing: Logistic Regression (Discriminative) ---
Success! Accuracy: 96.23%, Time: 0.06s
--- Testing: K-Neighbors (Distance-Based) ---
Success! Accuracy: 90.79%, Time: 0.14s
--- Testing: Gaussian NB (Wrong Assumption) ---
Success! Accuracy: 87.50%, Time: 0.44s
--- Testing: Linear Discriminant (LDA) ---
Success! Accuracy: 97.49%, Time: 61.56s
--- Testing: Quadratic Discriminant (QDA) ---
FAILED! Error: The covariance matrix of class 0 is not full rank. When using
`solver='svd'` the number of samples in each class should be more than the number
of features, but class 0 has 3377 samples and 6889 features. Try using
`solver='eigen'` and setting the parameter `shrinkage` for regularization.
```

Step 4: Show the Final Results

```
# Convert results to a pandas DataFrame for a "nice chart"
results_df = pd.DataFrame.from_dict(results, orient='index')
results_df = results_df.sort_values(by='Test Accuracy', ascending=False)

print("\n--- FINAL BAKE-OFF RANKING (Spam Filter) ---")
# We'll also show Time and Error to tell the full story
pd.set_option('display.max_colwidth', 100) # Show more of the error message
print(results_df)

# Plot the accuracy
plt.figure(figsize=(10, 6))
# We create a new column for plotting, setting failed models' accuracy to 0
plot_df = results_df.fillna(0)
sns.barplot(x=plot_df.index, y=plot_df['Test Accuracy'], hue=plot_df.index,
            palette='viridis', legend=False)
plt.xlabel('Model')
plt.ylabel('Test Set Accuracy')
plt.title('Final Model Comparison')
plt.xticks(rotation=45, ha='right')
plt.show()
```

--- FINAL BAKE-OFF RANKING (Spam Filter) ---

	Test Accuracy	Time (sec)	\
Multinomial NB (Text-Native)	0.987440	0.052082	
Linear Discriminant (LDA)	0.974880	61.555204	
Logistic Regression (Discriminative)	0.962321	0.064013	
K-Neighbors (Distance-Based)	0.907895	0.137356	
Gaussian NB (Wrong Assumption)	0.875000	0.438121	
Quadratic Discriminant (QDA)	0.000000	30.011519	

Error

Multinomial NB (Text-Native)

None

Linear Discriminant (LDA)

None

Logistic Regression (Discriminative)

None

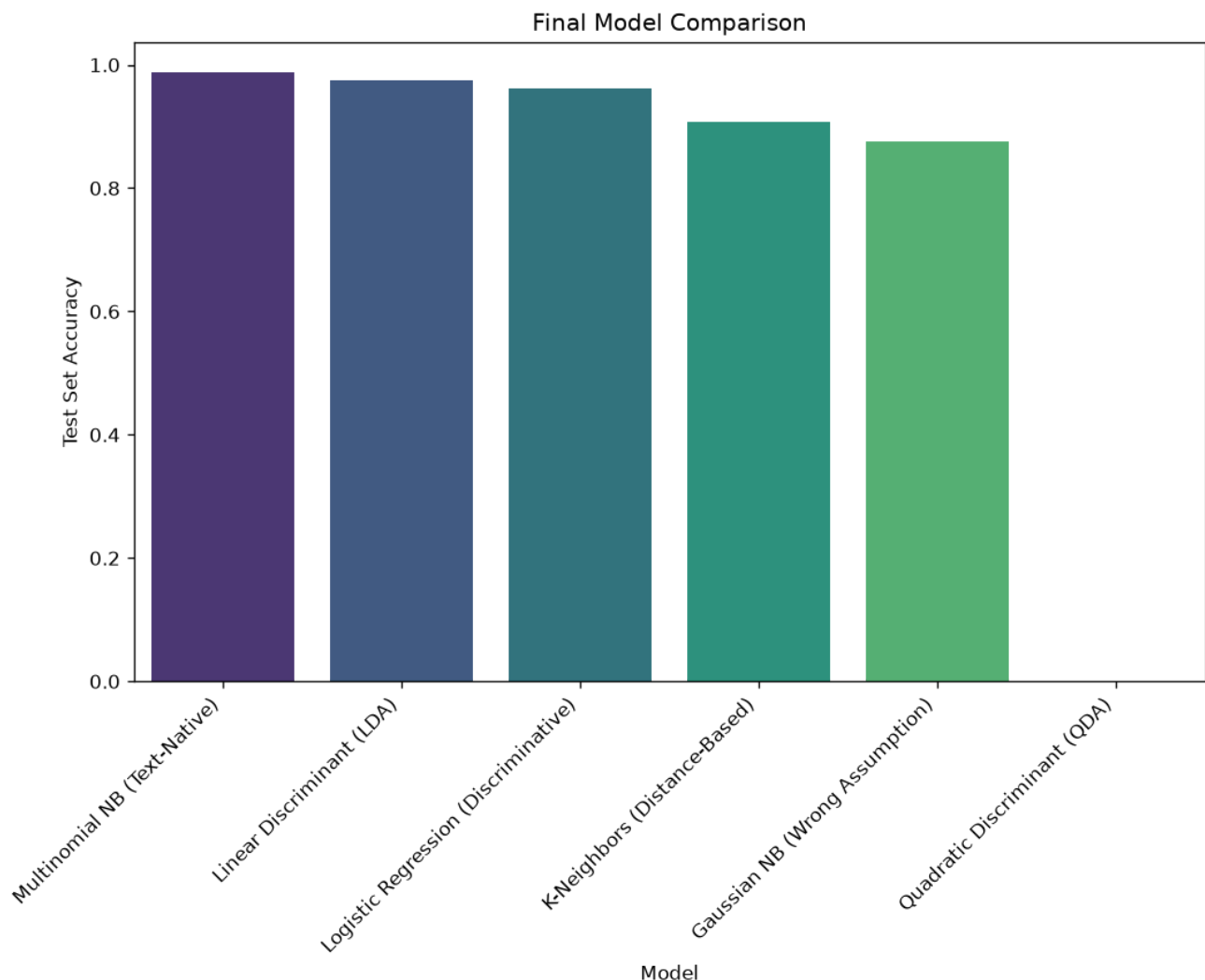
K-Neighbors (Distance-Based)

None

Gaussian NB (Wrong Assumption)

None

Quadratic Discriminant (QDA) The covariance matrix of class 0 is not full rank. When using `solver='svd'` the number of sampl...



Conclusion: The Right Tool for the Job

This bake-off tells a very clear story:

1. **Logistic Regression** and **MultinomialNB** were the **winners**. They are designed to handle high-dimensional, sparse text data and performed excellently.
2. **GaussianNB** performed poorly, but at least it ran. This is because our data is *counts* (mostly zeros), which does not follow a *Gaussian* (bell curve) distribution at all. This is a classic “Wrong Assumption” failure.
3. **KNeighborsClassifier** was also very inaccurate. This is a perfect demonstration of the **Curse of Dimensionality**. With over 7,000 features, the “distance” between any two messages becomes meaningless, so the “nearest neighbor” is no longer a reliable predictor.
4. **LDA** and **QDA** took a long time to run and gave warnings.

The final lesson is clear: You must choose a model that matches the assumptions and dimensionality of your data. For high-dimensional, sparse count data like text, **Multinomial Naive Bayes** and **Logistic Regression** are the correct, industry-standard tools.

Exercises

1. **Count the parameters.** For d features and K classes, write the number of parameters estimated by QDA, by LDA and by Gaussian naive Bayes. Evaluate each for $d = 10$, $K = 3$ and for $d = 1000$, $K = 3$.
2. **QDA against LDA.** Generate two classes with clearly different covariance matrices. Fit both models and plot the decision boundaries. Then shrink the difference between the covariances and watch the two boundaries converge.
3. **When naive is enough.** Compare Gaussian naive Bayes with LDA on a dataset with strongly correlated features. Naive Bayes is wrong about the correlations — does it classify worse? Explain the discrepancy.
4. **Priors matter.** Fit a generative classifier on an imbalanced dataset, then refit with `priors` set to uniform. How do the predictions change, and which choice is right if your training sample is not representative of the population?
5. **Zero counts.** Fit a multinomial naive Bayes on text with `alpha=0`. Find an input that produces a probability of exactly zero, and explain how Laplace smoothing fixes it.
6. **Generative or discriminative?** Compare naive Bayes and logistic regression on the same data at several training-set sizes. Which is better with very few examples, and which wins as the data grows?

References

- Naïve Bayes Classifier: https://en.wikipedia.org/wiki/Naive_Bayes_classifier;
- https://scikit-learn.org/stable/modules/lda_qda.html#lda-qda
- Section 4.4 of [1]

[1] James, Gareth Gareth Michael. An introduction to statistical learning: with applications in Python, 2023. <https://www.statlearning.com>